GIT-ONTO-LOGIC PROJECT

User Manual

Name: Andoni Karafilakis - Student Number: 24314657
Name: Kamleshkumar Senthilkumar - Student Number: 24245674

Table of contents

1. Introduction

The Git-Onto-Logic project is an application developed to allow users to understand key concepts of the git version control system by analysing a range of metadata from repositories. The application answers important questions about activities, contributors and patterns across repositories.

It uses an ontology built with **OWLReady2** to represent how core Git concepts (such as repositories, branches, commits, users, and files) are related. The ontology is populated with individuals using metadata extracted from multiple GitHub repositories using the GitHub REST API. With the addition of **SWRL rules** and **reasoning**, we are able to infer new knowledge and properties about the metadata allowing us to answer complex queries relating to the data.

This application is implemented using a **Flask interface** which enables users to view the metadata, traverse the knowledge graph's schema, and run SPARQL queries.

In the next section, we look at how to set up the environment and get the application running locally.

---

2. Environment setup guide

To properly run this application, a few setup steps need to be completed. Assuming the extraction of the zip file is complete, take a moment to review the folder structure. It contains the ontology, data, and application files that make up the system. Shown below is the folder structure:

```
CITS3005Project_24314657_24245674/
├── app/
│   └── app.py                          # Flask web interface
│
├── ontology/
│   ├── git_ontology_base.owl           # Base ontology (unpopulated)
│   ├── git_ontology_populated.owl      # Populated ontology (generated)
│   ├── git_ontology_reasoned.owl       # Reasoned ontology
│
├── data/                               # Extracted GitHub metadata
│   ├── branches.json
│   ├── commits.json
│   ├── files.json
│   ├── repos.json
```

```
│    └── users.json                          # Extracted GitHub metadata
│
├── docs/                                    # User manual and reports
│    ├── user_manual.pdf
│    ├── 24245674_individual_report.pdf
│    └── 24314657_individual_report.pdf
│
├── scripts/                  # Ontology population and reasoning script
│    └── populate_and_reasoning.py
│
├── requirements.txt                         # Python dependencies
└── README.md                                # Quick guide to instructions
```

2.1. Creating and activating a virtual environment

We will now create a virtual environment and install all our dependencies required to run this project. Make sure that you are in the right directory (**CITS3005Project_24314657_24245674**).

1. Create and Activate Virtual Environment

```
# Create virtual environment
python -m venv venv

# On windows
> venv\Scripts\activate

# On Linux/Mac
$ source venv/bin/activate
```

2. Install Dependencies

```
$ pip install -r requirements.txt
```

In the next section we will walk you through populating, reasoning and loading the ontology using python script **populate_and reasoning.py**.

2.2. Loading our ontology

In this section, we load, populate and reason our ontology. First, we will explain the structure of the OWL files used and generated in this project. The ontology is organised into three main files that represent different stages of processing.

1. The **`git_ontology_base.owl`** file contains the unpopulated ontology. It was created and edited in *Protégé*, where all classes, object properties, data properties, and SWRL rules were defined.
2. The **`git_ontology_populated.owl`** file is produced[1] after populating the ontology with individuals from the GitHub metadata stored in the JSON files located in the `data/` folder.
3. Finally, the **`git_ontology_reasoned.owl`** file contains the fully reasoned knowledge graph. This version includes all inferred relationships and classifications after applying SWRL rules and running the reasoner.

Keeping these three files separate allows users to easily repopulate the ontology using different metadata, and to inspect the ontology before and after reasoning.

The population and reasoning of the ontology can be completed by executing the following commands in your terminal.

1. By running this python script, 2 owl files as mentioned above will be generated. Please allow 1-3 minutes for the population and reasoning to complete.

```
$ python scripts/populate_and_reasoning.py
```

2. Next, to start the web application, execute:

```
$ python app/app.py
```

3. Allow a moment for the application to boot up. Once you see “Debugger is active!”, Ctrl + Click on the local host link shown below or paste into the browser.

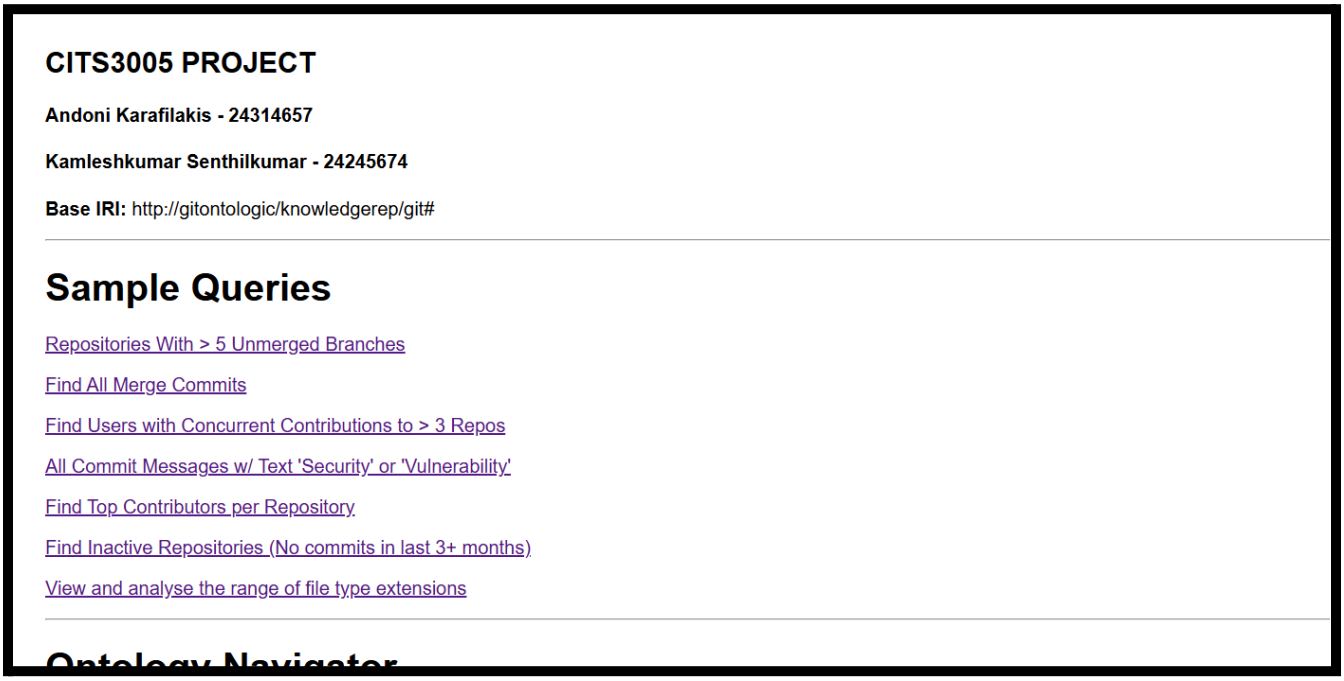
```
http://127.0.0.1:5000
```

You have accessed the web interface! It should look like the *Image1* below .

---

[1] We have provided the **git_ontology_reasoned.owl** file. In case the Pellet reasoner times out or hangs due to system differences, the file is readily available for use. If that happens, skip step 1 and continue from step 2. However, the standard procedure is to run the **populate_and_reasoning.py** script, which will recreate the file on the spot.

3. Navigating the web interface



Images 1, 2: Images showing the home page of the flask web interface.

This section will guide you through navigating the web interface. When you first start up the home page you will see the following sections:

1. Sample Queries
2. Ontology Navigator
3. Search Feature
4. Data Validation
5. Data Management

Simply click on the various hyperlinks on the pages to navigate through the app. If you would like to return back to a page you had just left or return to the home page, there are navigation links in the top left corner of the page.

Sample Queries:
Clicking on any of the hyperlinks below the heading will run the SPARQL query with the purpose described in the link name as well as in more detail on the subsequent page of results.

Ontology Navigator:
Click on the link below the heading to enter the data browsing section. Here you are able to explore instances of every inferred class and relation, including subclasses.

Search Feature:
Click on the link below the ‘Search Feature’ heading to enter the searching section. Follow the instructions in there to formulate your own custom searches of the data using SPARQL queries. In the Select field, you place which columns (e.g. ?repo ?branch) you want to include in your output column, in the where column you include your constraints (relations and filters) and you also have the option to Order the results by a certain column (e.g. ?commitDate) or filter your results using a certain subclass (e.g. InitialCommit).

Data Validation:
Click on the link below the ‘Data Validation’ heading to check for any errors in the data according to the ontology. You will see any errors classified as either “Critical” or “Warning” depending on how influential we believe the error in the data is. There is also a summary of the counts of all the different classes below.

Data Management:
By clicking on the hyperlink found below the ‘data management’ heading, you are able to navigate through the menu and add, update, and delete data in the knowledge graph. Simply click on the hyperlink that corresponds to which operation you want to do. More detailed instructions on how to do so can be found on the page, as well as in the ‘adding, updating and removing data’ section below.

4. Schema of ontology

This section discusses the overall design of the **ontology**, classes, subclasses, object properties, data properties, SWRL rules and the reasoner. This ontology models the domain of the Git **version control system**. *Figure1*, a simple **UML-style** schematic representation of the knowledge graph is shown below.

The **object properties shown in black** represent direct relationships, **red** represents inverse relationships, and **blue** indicates inferred relationships. The **subclass hierarchy** is represented with hollow arrows pointing towards their main (parent) classes.

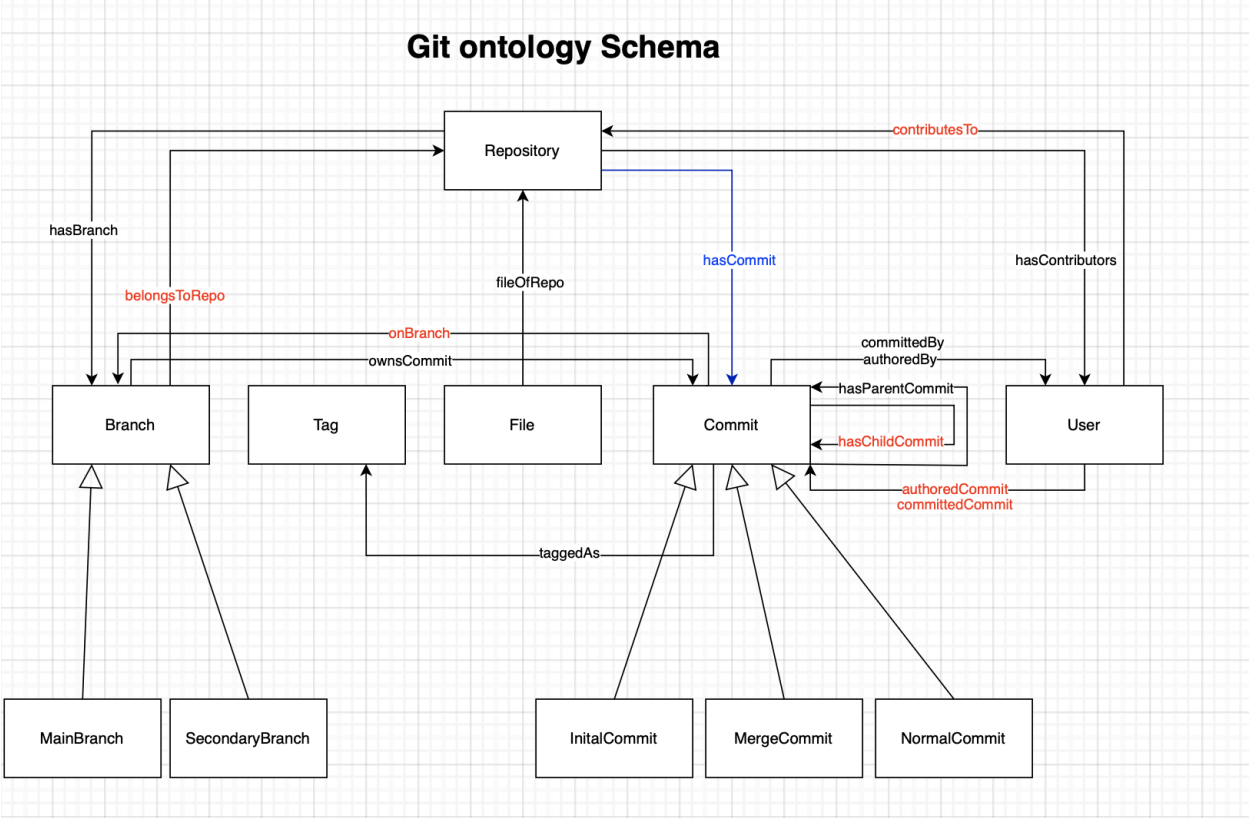


Figure 1: UML-style representation of the ontology visually representing all classes, relationships between them and inferred property.

Next, sections 4.1 - 4.4 will outline the designs of the ontology in more detail. Classes, subclasses, object properties, data properties, and the SWRL rules that enable reasoning and inference. The specific design choices and justifications are discussed further in the individual reports.

4.1. Classes, subclasses and axioms

Table 1: Shows the classes, subclasses, axioms and descriptions.

Class	**Subclass**	**Axioms / characteristics**	**Disjoints / equivalences**	**Description**
Repository	--	hasBranch **min** 1 Branch hasContributors **some** User	--	Represents a Git repository which stores files, branches and contributors.
Branch	**MainBranch** (default branch); **SecondaryBranch** (non-default branch)	belongsToRepo **some** Repository ownsCommit **some** Commit	Disjoint with: MainBranch, SecondaryBranch	Represents an independent working line in a repository
Commit	**InitialCommit** (exactly 0 parentCommit); **NormalCommit** (exactly 1 parentCommit); **MergeCommit** (exactly 2 parentCommit).	authoredBy **some** User committedBy **some** User onBranch **some** Branch taggedAs **min** 0 Tag	InitialCommit ≡ Commit **and** (hasParentCommit **exactly** 0 Commit) **and** (is_initial **value** true); NormalCommit ≡ Commit **and** (hasParentCommit exactly 1 Commit) **and** (is_initial **value** false); MergeCommit ≡ Commit **and** (hasParentCommit **exactly** 2 Commit);	Represents a snapshot of a change made to the Git repository
User	--	contributesTo **some** Repository	--	Represents contributors to a project/repository.
File	--	fileOfRepo **some** Repository	--	Represents files in a repository
Tag	--	--	--	Represents labels added to commits

4.2. Object properties

Table 2: Shows object properties with domains, inverses, characteristics, and descriptions.

Property	Domain → Range	Inverse	Characteristics / Chains	Description
`hasBranch`	Repository → Branch	`belongsToRepo`	**--**	Connects a repository to its branches.
`belongsToRepo`	Branch → Repository	`hasBranch`	Functional	Links each branch to its parent repository.
`ownsCommit`	Branch → Commit	`onBranch`	--	Defines commits owned by a branch.
`onBranch`	Commit → Branch	`ownsCommit`	--	Connects a commit to the branch it was made from.
`hasCommit`	Repository → Commit	--	hasBranch **o** ownsCommit SubPropertyOf: hasCommit	Links the commit to the repository it was made in.
`hasParentCommit`	Commit → Commit	`hasChildCommit`	Irreflexive	For ancestry/order between commits.
`hasChildCommit`	Commit → Commit	`hasParentCommit`	Irreflexive	For ancestry/order between commits.
`authoredBy`	Commit → User	`authoredCommit`	Functional Irreflexive	A commit is authored by a single user
`committedBy`	Commit → User	`committedCommit`	Functional Irreflexive	A commit is committed by a single user
`contributesTo`	User → Repository	`hasContributors`	--	Links all repositories contributed by a user
`hasContributors`	Repository → User	`contributesTo`	--	All users who contributed to a repository
`fileOfRepo`	File → Repository	--	Functional	Connects file to the repository it belongs to
`taggedAs`	Commit → Tag	--	--	Indicates labels made by a commit

4.3. Data properties

*Table 3,4,5,6: This section outlines the da*tatype properties grouped by their class.

Repository

Property	Range	Characteristic
repo_id	xsd:int	Functional
repo_name	xsd:string	Functional
repo_url	xsd:string	Functional
repo_description	xsd:string	Functional
repo_language	xsd:string	Functional
repo_stars	xsd:int	Functional
repo_forks	xsd:int	Functional

Branch

Property	Range	Characteristic
branch_name	xsd:string	Functional
is_default	xsd:boolean	Functional

User

Property	Range	Characteristic
user_id	xsd:integer	Functional
user_login	xsd:string	Functional
user_url	xsd:string	Functional

Commit

Property	Range	Characteristic
commit_sha	xsd:string	Functional
commit_message	xsd:string	Functional
commit_date	xsd:dateTimeStamp	Functional
commit_author_login	xsd:string	Functional
commit_committer_login	xsd:string	Functional
commit_parent_count	xsd:int	Functional
commit_parents	xsd:string	--
is_initial	xsd:boolean	Functional

File

Property	Range	Characteristic
file_name	xsd:string	Functional
file_status	xsd:string	Functional
file_additions	xsd:int	Functional
file_deletions	xsd:int	Functional
file_changes	xsd:int	Functional

Tag

Property	Range	Characteristic
commit_tag	xsd:string	Functional

4.4. SWRL rules and inference capabilities

Tables 7, 8, 9, 10, 11, 12, 13: SWRL rules used in the ontology and the resulting inferred relations.

Rule name R1_UserContributesToRepository	**Description/ inference result** If a commit is made by a user on a branch belonging to a repository, then that user contributes to that repository.
SWRL Expression Commit(?c) ^ committedBy(?c, ?u) ^ onBranch(?c, ?b) ^ belongsToRepo(?b, ?r) -> contributesTo(?u, ?r)	

Rule name R2_ParentChildCommit	**Description/ inference result** If a commit has a parent, that parent is inferred to have the commit as a child.
SWRL Expression hasParentCommit(?c1, ?c2) -> hasChildCommit(?c2, ?c1)	

Rule name R3_MergeCommit	**Description/ inference result** A commit with two or more parent commits is classified as a MergeCommit.
SWRL Expression Commit(?c) ^ commit_parent_count(?c, ?n) ^ swrlb:equal(?n, 2) -> MergeCommit(?c)	

Rule name R4_InitalCommit	**Description/ inference result** A commit with no parent is inferred as an InitialCommit.
SWRL Expression Commit(?c) ^ is_initial(?c, true) ^ commit_parent_count(?c, ?n) ^ swrlb:equal(?n, 0) -> InitialCommit(?c)	

Rule name R5_NormalCommit	**Description/ inference result** A commit with exactly one parent is classified as a NormalCommit.
SWRL Expression Commit(?c) ^ commit_parent_count(?c, ?n) ^ swrlb:equal(?n, 1) -> NormalCommit(?c)	

Rule name R6_MainBranch	**Description/ inference result** Classifies a branch as MainBranch when its *is_default* property is true.
SWRL Expression Branch(?b) ^ is_default(?b, true) -> MainBranch(?b)	

Rule name R7_SecondaryBranch	**Description/ inference result** Classifies a branch as SecondaryBranch when *is_default* is false.
SWRL Expression Branch(?b) ^ is_default(?b, false) -> SecondaryBranch(?b)	

These SWRL rules allow for the proper classification of individuals into classes and subclasses based on their data properties. Rules R3_MergeCommit, R4_InitalCommit and R5_NormalCommit allow commits to be inferred as MergeCommit, InitialCommit or NormalCommits based on the commit_parent_count data property value. Similarly, rules R6_MainBranch and R7_SecondaryBranch allow branches to be inferred as MainBranch or SecondaryBranch based on the is_default data property value.

These rules, along with the preexisting axioms, are processed by the **Pellet reasoner** to produce inferred triples, which are subsequently stored in the reasoned ontology. These inferred associations can be searched via SPARQL and are displayed in the Flask interface. The choice behind the specific reasoner is discussed in the individual report.

A key transitive property used in the ontology is the parent-child commit relationship. This is implemented with the **hasParentCommit** and **hasChildCommit** object properties. They are inverses of each other and allows for the traversal of commit history in bi-directionally. This enables reasoning over complex commit chains.

Additionally, a **property chain axiom** is defined for commit relationships: hasBranch o ownsCommit ⊑ hasCommit. This axiom allows the reasoner to infer that if a repository *hasBranch* and that branch *ownsCommit*, then the repository *hasCommit*. This property chain maintains logical consistency within the ontology and helps streamline query performance.

5. Example Queries

All of the 7 queries can be found on the home page of the web application. Form them by clicking on their respective hyperlink. Below is an explanation of the 4 required sample queries along with the format of their output.

Find all repositories with more than 5 unmerged branches

```
query = '''
PREFIX git: <http://gitontologic/knowledgerep/git#>
SELECT ?Repository (COUNT(?Branch) AS ?branchCount)
WHERE {
    ?Branch a git:Branch.
    ?Branch git:belongsToRepo ?Repository .
}
GROUP BY ?Repository
HAVING (COUNT(?Branch) > 5)
ORDER BY DESC(?branchCount)
'''
```

Forms a query by selecting repositories and counting how many branches each has, then showing only those with more than five branches, sorted from most to least.

Query: Repositories with >5 unmerged branches

Repository	Branch Count
repo_306116800	23
repo_29428349	6

Find all users who have made concurrent contributions to 3 or more repos

```
query = '''
PREFIX git: <http://gitontologic/knowledgerep/git#>
SELECT ?user ?yearMonth (COUNT(DISTINCT ?repo) AS ?repoCount)
(GROUP_CONCAT(DISTINCT ?repoName; separator=", ") AS ?repositories)
WHERE {
    ?commit a git:Commit .
    ?commit git:authoredBy ?user .
    ?commit git:onBranch ?branch.
    ?commit git:commit_date ?commitDate .
    ?branch git:belongsToRepo ?repo .
    ?repo git:repo_name ?repoName .

    # need to take out the month part of the date
    BIND(SUBSTR(STR(?commitDate), 1, 7) AS ?yearMonth)
}
GROUP BY ?user ?yearMonth
HAVING (COUNT(DISTINCT ?repo) >= 3)
ORDER BY DESC(?repoCount) ?user ?yearMonth
'''
```

Forms a query by looking at all commits, linking each to its author, branch and repo, and reads the commit date. For each user and month, it counts how many distinct repositories they committed to and collects the repository names. It only keeps the user-months with at least 3 repos.

Query: Users with Concurrent Contributions to 3+ Repositories

Concurrent = contributions made in the same month

Found 42 user-month combinations with 3+ repository contributions:

User	Month	Repo Count	Repository Names
user_dependabot[bot]	2025-09	7	pytest-dev/pytest-reportlog, pytest-dev/pytest-repeat, pytest-dev/pytest-services, pytest-dev/pytest-plus, pytest-dev/execnet, pytest-dev/pytest-order, pytest-dev/pytest-random-order
user_pre-commit-ci[bot]	2025-10	7	pytest-dev/pytest-base-url, pytest-dev/pytest-variables, pytest-dev/pytest-reportlog, pytest-dev/pytest-metadata, pytest-dev/execnet, pytest-dev/pytest-order, pytest-dev/pytest-random-order
user_pre-commit-ci[bot]	2025-08	6	pytest-dev/pytest-base-url, pytest-dev/pytest-variables, pytest-dev/pytest-reportlog, pytest-dev/execnet, pytest-dev/pytest-order, pytest-dev/pytest-random-order
user_pre-commit-ci[bot]	2025-09	6	pytest-dev/pytest-base-url, pytest-dev/pytest-variables, pytest-dev/pytest-reportlog, pytest-dev/execnet, pytest-dev/pytest-order, pytest-dev/pytest-random-order

Find all commits that are merges

```
query = '''
PREFIX git: <http://gitontologic/knowledgerep/git#>
SELECT ?commit ?commitMessage ?parentCount ?branch ?commitDate ?author
WHERE {
    ?commit a git:Commit .
    ?commit git:commit_parent_count ?parentCount .
    ?commit git:commit_message ?commitMessage .
    ?commit git:onBranch ?branch .
    ?commit git:commit_date ?commitDate .
    ?commit git:authoredBy ?author .
    FILTER(?parentCount > 1)
}
ORDER BY DESC(?commitDate)
'''
```

Forms a query by selecting commits with more than one parent (which makes them merge commits), showing the commit, message, parent count, branch, date, and author. It then orders them from newest first.

Query: All Merge Commits

Found 445 merge commit(s):

Commit	Message	Parent Count	Branch	Date	Author
commit_681185564_f2ab02597b0f660432c52da924db0df64c4616a4	Merge pull request #6 from pytest-dev/5-fake-issue Fix fake...	2	branch_681185564__main	2025-07-25	user_azmeuk
commit_66012563_4ba2216e0b29e794601d67208a172c50dd0b127d	Merge pull request #65 from tn3w/fix-linter-errors Remove u...	2	branch_66012563__main	2025-07-16	user_RonnyPfannschr
commit_29428349_07c42e6781b6a77030305e4f721126523a89bb06	Changelog for release 2.2.2 (#51)	2	branch_29428349__main	2025-07-16	user_nicoddemus
commit_77096478_7404a12ae1b88a339c78fcdb97cfb386fdbc58cf	Merge pull request #62 from pytest-dev/release-1.2.0 Releas...	2	branch_77096478__pre_commit_ci_update_config	2025-06-22	user_nicoddemus
commit_77096478_754a8e45b848a86f236d6e079186e0d55d3ab7be	Remove `return` from `finally` block (#60) * Remove `return...	2	branch_77096478__release_1_2_0	2025-06-22	user_nicoddemus

Flag all commits messages containing the text “security” or “vulnerability”

```
query = '''
PREFIX git: <http://gitontologic/knowledgerep/git#>
SELECT ?commit ?commitMessage ?branch ?author ?commitDate ?repo ?repoName
WHERE {
    ?commit a git:Commit .
    ?commit git:commit_message ?commitMessage .
    ?commit git:onBranch ?branch .
    ?commit git:authoredBy ?author .
    ?commit git:commit_date ?commitDate .
    ?branch git:belongsToRepo ?repo .
    ?repo git:repo_name ?repoName .

    # check for security stuff - case insensitve
    FILTER(CONTAINS(LCASE(?commitMessage), "security") || CONTAINS(LCASE(?commitMessage), "vulnerability"))
}
ORDER BY DESC(?commitDate)
'''
```

Forms a query by finding commits whose message contains “security” or “vulnerability”, links each to its branch and repository to show names, and returns commit, message, branch, author, date, and repo. The result is ordered in descending order, from newest commit first.

Security-Related Commits

Searching for commits containing: security or vulnerability

Found 4 security-related commit(s):

Commit	Message	Branch	Author	Date	Repository
commit_681185564_ade288ec47bf52b785603f997e7d311ae4a3647a	fix: joserfc security warning	branch_681185564__main	user_azmeuk	2025-08-26T08:34	pytest-dev/pytest-iam
commit_97933450_7d32688324d98a35798ccacf6f923b5195526404	Merge pull request #323 from bluetech/update-action ci: update actions/download-artifact to fix vulnerability	branch_97933450__master	user_RonnyPfannschmidt	2025-01-07T12:41	pytest-dev/execnet
commit_97933450_2f4578fd5329751e86147c5a9420fddc62da8ba2	ci: update actions/download-artifact to fix vulnerability Fix this vulnerability: https://github.com/advisories/GHSA-6q32-hq47-5qq3	branch_97933450__master	user_bluetech	2025-01-07T10:31	pytest-dev/execnet
commit_44554687_f6da17158bc0dc6c2c022a26dcc79f39ba3415cf	Keep GitHub Actions up to date with GitHub's Dependabot (#80) Like #78 but automated. https://docs.github.com/en/code-security/dependabot/working-with-dependabot/keeping-your-act... Commit Message Too Long To Display	branch_44554687__release_0_9_4	user_cclauss	2024-01-23T04:52	pytest-dev/pytest-repeat

6. Adding, Updating and Removing Data:

Our web app allows for the addition, update and removal of data in the knowledge graph. It is made very simple in the user interface with clear instructions in the “Data Management” section. Note that in order to view any additions, updates and deletes made to the knowledge graph, the web app needs to be restarted in order to reload the data.

- **Adding** new data is done by filling out a form specific to which type of information you would like to add (user, repository, commit, file, branch) You simply input the requested details about the object and it is inserted into the knowledge graph by building a SPARQL INSERT query using f strings and the inputs from the form.

- **Updating** new data is done by filling out a form with the URI of the entity you want to update, the property to update and the old and new values. It works by using a combination of DELETE and INSERT in SPARQL to remove the existing value and insert a new one in its place, again using f strings.

- **Removing** data is done similarly to adding and updating where the user fills out a delete form with entity, property and value URIs that can be adjusted to meet certain requirements. The knowledge graph is updated using SPARQL DELETE queries.

---

7. How To Add Rules To The Ontology:

Rules can be added to the Ontology in the populate_and_reasoning.py script that populates the ontology with the real dataset and applies reasoning. There is a section in the script that we have identified with comments to demonstrate where rules can be added. This can be done using SWRL rules in OWLReady2, using IMP and *set_as_rule(...)*. The Imp (implication) is an OWLReady 2 class that holds a specific SWRL rule. *set_as_rule(...)* attaches the SWRL rule text (written in SWRL syntax) to the Imp. After this is added, the reasoner applies the added rules. Below is an example of a rule that could be added to the ontology. Note that Owlready2 does not support the SWRL built-ins. Standard SWRL syntax should be used.

```
with onto:
    r = Imp("MainBranchRule")
    r.set_as_rule("""
        Branch(?b) ^ is_default(?b,true)
        -> MainBranch(?b)
    """)
```

References:

Ontologies with Python - Programming OWL 2.0 Ontologies with Python and Owlready2
https://docs.github.com/en/rest
https://kgbook.org/#sec-preface
https://www.w3.org/submissions/SWRL/
https://owlready2.readthedocs.io/en/v0.48/
https://github.com/pytest-dev